

Registered Framework-Dynamics Visualization Specification

Plot-D1/D2 mean framework geometry and selected atomic trajectories

mdstats

Contents

Purpose and status	1
Motive	2
Normative ownership	2
Public modules and exports	2
Input semantics	3
Frame collection	3
Framework topology or catalog	3
Selected frames	3
Periodic gauge normalization	3
Display cell	3
Registration modes	4
Material coordinates	4
Laboratory coordinates	4
Framework-registered coordinates	4
Geometrical mean framework	4
Atom selection	5
Continuous trajectory paths	5
Folded trajectory paths	5
Public preparation API	5
Prepared result model	6
TrajectoryPathSet	6
FrameworkDynamicsScene	6
Partitioned topology categories	6
Renderer	7
Category legend groups	7
Interactive density scene fitting	8
Progress reporting	8
Resource policy	8
Failure semantics	8
Numerical and semantic limitations	8
External methods and provenance	9
Focused validation	9
Plot-D3 and Plot-D4 density extensions	9
Example topology-inference boundary	10

Purpose and status

This document is the normative specification for Plot-D1 and Plot-D2 of the framework-dynamics visualization extension, introduced in mdstats 0.19.30a0 and updated through the LD10 runtime-resource integration in mdstats 0.19.64a0. The implementation prepares a topology-compatible geometrical mean of a projected framework or its retained atomic paths and overlays the trajectories of selected atoms in the same registered coordinate system.

The implementation is deliberately divided into two layers:

1. `prepare_framework_dynamics_scene(...)` performs coordinate registration, periodic gauge normalization, framework averaging, atom selection, path construction, and resource preflight without importing Plotly.
2. `plot_framework_dynamics_3d(...)` adds the prepared paths to the existing generic Plotly graph renderer.

Atomic-density fields are specified in `atomic_density_spec.md`; framework vertex and edge density fields are specified in `framework_density_spec.md`. Both reuse the coordinate-registration and mean-framework boundary specified here.

Motive

A static framework graph is an important structural cue, but it cannot directly show whether a mobile ion remains localized, circulates around a ring, or hops between distant sites. Conversely, raw trajectories are hard to interpret without the framework in which they occur. The desired diagnostic scene is therefore

registered mean framework + selected atomic paths .

The same registration boundary will later support time- or ensemble-averaged atomic and framework densities. Defining it first prevents trajectory paths, density clouds, and framework geometry from being accumulated in mutually incompatible periodic gauges.

Normative ownership

This specification owns:

- selected-frame validation;
- topology-compatible frame-local framework adaptation;
- deterministic normalization of graph-node image gauges;
- reference-cell, mean-cell, material, laboratory, and framework-registered coordinate policies;
- the geometrical mean framework view;
- atom-index and species selection;
- continuous and folded trajectory representations;
- explicit folded-path breaks at periodic-cell crossings;
- preparation and rendering resource controls;
- composition with the existing Plotly 3-D graph renderer;
- HTML export of the composite scene.

It does not own:

- atomic connectivity or framework projection;
- topology-state discovery or segmentation;
- trajectory unwrapping performed by the readers;
- atomic or framework density estimation;
- ring-site or cage assignment;
- hop detection or kinetic-state classification;
- generic graph styling or periodic graph materialization.

Public modules and exports

`mdstats/plotting/framework_dynamics.py`

The following symbols are exported from `mdstats.plotting` and the package root:

```
SpatialRegistrationMode
TrajectoryDisplayMode
FrameworkDynamicsOptions
FrameworkDynamicsResources
TrajectoryAtomSelection
TrajectoryPathSet
FrameworkDynamicsScene
Trajectory3DRenderOptions
FrameworkDynamicsRenderResult
prepare_framework_dynamics_scene
plot_framework_dynamics_3d
```

Input semantics

Frame collection

collection must be an `AtomisticFrameCollection` with fixed atom ordering. Mean-framework preparation accepts either trajectory or ensemble semantics. Atomic trajectory preparation requires

```
collection.require_trajectory("atomic trajectory visualization")
```

and therefore rejects an independent ensemble.

Framework topology or catalog

topology must be one authoritative `FrameworkTopology` or one compatible `TopologyCatalog`. A single topology applies one exact graph to all selected frames. A catalog supplies exact topology classes and frame membership; this module never reconstructs or approximately matches topology classes.

Every selected category must preserve:

- framework node identity;
- edge keys and periodic winding;
- atomic ordering and atomic numbers;
- periodic boundary flags;
- mapping and catalog provenance.

A selected frame absent from the catalog, an incompatible mapping, or incompatible node identity raises `GraphAdapterError`.

Selected frames

`frame_indices` is a nonempty, unique, strictly increasing sequence of collection positions. If omitted, all frames are selected. `reference_frame`, when provided, must belong to the selected sequence.

Periodic gauge normalization

The existing framework adapter supplies wrapped node positions and one image shift for every displayed edge. For frame t , let the wrapped fractional node coordinate be $\mathbf{f}_v^w(t)$, and let an oriented graph edge $e = (u, v)$ carry shift $\mathbf{n}_e(t) \in \mathbb{Z}^3$.

A deterministic spanning forest assigns an integer node gauge $\mathbf{g}_v(t)$. For every tree edge,

$$\mathbf{g}_v(t) = \mathbf{g}_u(t) + \mathbf{n}_e(t).$$

The lifted node coordinate is

$$\mathbf{f}_v^L(t) = \mathbf{f}_v^w(t) + \mathbf{g}_v(t) + \mathbf{q}_{C(v)}(t),$$

where $C(v)$ is the connected component of node v and $\mathbf{q}_C(t) \in \mathbb{Z}^3$ is its component shift. For a trajectory, each $\mathbf{q}_C(t)$ is chosen so that the lowest-index node of that component agrees with its reader-supplied unwrapped fractional coordinate. For an ensemble, each component anchor is placed independently in the canonical cell.

The residual edge voltage is

$$\tilde{\mathbf{n}}_e = \mathbf{n}_e(t) + \mathbf{g}_u(t) - \mathbf{g}_v(t).$$

It must be identical in every selected frame. Tree-edge residuals may be zero; non-tree periodic winding remains in $\tilde{\mathbf{n}}_e$. This change of gauge does not change the scientific periodic graph.

Display cell

The display cell H_d is selected by `display_cell`:

"reference" Use the cell of `reference_frame`.

"mean" Use the arithmetic mean of the selected finite cell matrices, $H_d = N^{-1} \sum_t H_t$.

The selected matrix must be finite and nonsingular. The mean-cell option is a visual reference and is not asserted to be a thermodynamic mean strain measure.

Registration modes

Material coordinates

Material registration maps lifted fractional coordinates into one display cell:

$$\mathbf{x}_v^{\text{mat}}(t) = \mathbf{f}_v^{\text{L}}(t)H_{\text{d}}.$$

Homogeneous cell expansion, contraction, and shear do not appear as internal framework motion. This is the default and is generally the most useful mode for site-hopping diagnostics.

Laboratory coordinates

Laboratory registration preserves each instantaneous cell:

$$\mathbf{x}_v^{\text{lab}}(t) = \mathbf{f}_v^{\text{L}}(t)H_t.$$

It displays physical homogeneous deformation together with internal motion. The mean graph is still decorated with the chosen display cell for periodic rendering; therefore the average is descriptive rather than an exact instantaneous graph at any one frame.

Framework-registered coordinates

Let the lifted framework centroid be

$$\mathbf{c}(t) = \frac{1}{N_V} \sum_{v=1}^{N_V} \mathbf{f}_v^{\text{L}}(t).$$

The residual translational drift relative to the first selected frame is

$$\Delta\mathbf{c}(t) = \mathbf{c}(t) - \mathbf{c}(t_0).$$

Framework-registered coordinates are

$$\mathbf{f}_v^{\text{reg}}(t) = \mathbf{f}_v^{\text{L}}(t) - \Delta\mathbf{c}(t).$$

The same $\Delta\mathbf{c}(t)$ is subtracted from every selected atomic trajectory. This removes only translation. It does not perform rotational or affine best-fit alignment.

Geometrical mean framework

Selected frames use uniform weights

$$w_t = \frac{1}{N}, \quad \sum_t w_t = 1.$$

After the chosen registration, the mean node position is

$$\bar{\mathbf{x}}_v = \sum_t w_t \mathbf{x}_v(t).$$

The output is one immutable DecoratedGraphView with:

- the reference node and edge keys;
- the reference node and edge attributes;
- mean registered node positions;
- normalized residual edge shifts;
- the chosen display cell;
- source frame and registration metadata.

The mean view is a visualization object. It does not replace the authoritative FrameworkTopology and is not accepted as a new scientific topology.

Atom selection

```
TrajectoryAtomSelection(  
    atom_indices=(...),  
    species=(...),  
    label=...,  
)
```

The selected atom set is the sorted union of:

- explicit nonnegative collection atom indices;
- every atom whose atomic number matches a supplied chemical symbol or atomic number.

The resolved selection must be nonempty and within the collection. This provides individual-atom, arbitrary-group, and whole-species selection without introducing separate APIs.

Continuous trajectory paths

The reader-owned trajectory fractional coordinates are already time-unwrapped. For selected atom i , material coordinates are

$$\mathbf{r}_i(t) = \mathbf{f}_i(t)H_d,$$

or, in framework-registered mode,

$$\mathbf{r}_i^{\text{reg}}(t) = [\mathbf{f}_i(t) - \Delta\mathbf{c}(t)]H_d.$$

Laboratory mode instead uses the instantaneous cell H_t . A continuous path is rendered as consecutive line segments and may extend across multiple periodic images.

Folded trajectory paths

For periodic axis α , define

$$\mathbf{m}_i(t)_\alpha = \lfloor \mathbf{f}_i(t)_\alpha \rfloor, \quad \mathbf{f}_i^{\text{fold}}(t)_\alpha = \mathbf{f}_i(t)_\alpha - \mathbf{m}_i(t)_\alpha.$$

Nonperiodic components are unchanged. The folded display coordinate is

$$\mathbf{r}_i^{\text{fold}}(t) = \mathbf{f}_i^{\text{fold}}(t)H_d.$$

A segment between frames t and $t + 1$ is broken when

$$\mathbf{m}_i(t + 1) \neq \mathbf{m}_i(t)$$

on any periodic axis. The renderer inserts None separators rather than drawing a false long diagonal across the unit cell.

Public preparation API

```
def prepare_framework_dynamics_scene(  
    collection: AtomisticFrameCollection,  
    topology: FrameworkTopology | TopologyCatalog,  
    *,  
    frame_indices: Sequence[int] | None = None,  
    display_mode: FrameworkGraphDisplayMode | str = "projected",  
    trajectory_selection: TrajectoryAtomSelection | None = None,  
    options: FrameworkDynamicsOptions | None = None,  
    resources: FrameworkDynamicsResources | None = None,  
    progress: ProgressPortLike | None = None,  
    progress_callback: Callable[[str], None] | None = None,  
) -> FrameworkDynamicsScene:  
    ...
```

display_mode may be projected or atomic_paths. Every selected frame uses the same mode.

Prepared result model

TrajectoryPathSet

Stores:

- resolved atom indices and atomic numbers;
- selected collection frame positions and frame IDs;
- physical times when available;
- continuous positions;
- displayed positions;
- lattice image vectors;
- display-segment break flags;
- selection label and path mode.

All numerical arrays are defensive, C-contiguous, finite where applicable, and read-only.

FrameworkDynamicsScene

Stores:

- the dominant registered mean framework DecoratedGraphView;
- zero or one global TrajectoryPathSet;
- the dominant atomic mean-connectivity graph when requested;
- selected frames and normalized global weights;
- display cell, options, and resource policy;
- global atomic and framework density fields;
- zero or more FrameworkTopologyCategoryLayer records;
- the source TopologyCatalog when category preparation is used;
- dominant_topology_id, planning records, and collection provenance.

Multiple scientific atom selections may be united in one TrajectoryAtomSelection; rendering-only groups must not duplicate the underlying paths or density fields.

Partitioned topology categories

prepare_framework_dynamics_scene accepts

FrameworkTopology | TopologyCatalog

A single topology is the one-category special case. A catalog produces one immutable FrameworkTopologyCategoryLayer per selected topology class:

```
FrameworkTopologyCategoryLayer(  
    topology_id,  
    topology,  
    frame_indices,  
    probability,  
    segment_count,  
    mean_framework,  
    atomic_mean_graph,  
)
```

All selected frames are first normalized into one global periodic gauge and display cell. The scene then computes, for each category:

1. the intersection of selected frames with the catalog group;
2. category-local normalized weights;
3. a category-conditioned mean framework;
4. a category-conditioned atomic mean-connectivity graph;
5. population and segment metadata.

Trajectories, atomic density fields, and framework density fields remain global by default and are not multiplied by the number of categories. The most populated category supplies the backward-compatible `mean_framework` and `atomic_mean_graph` fields and is recorded as `dominant_topology_id`.

The category frame sets must be nonempty, disjoint, and cover the selected catalog frames. Their probabilities must sum to one within numerical tolerance.

Renderer

```
def plot_framework_dynamics_3d(
    scene: FrameworkDynamicsScene,
    *,
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    graph_options: Graph3DRenderOptions | None = None,
    trajectory_options: Trajectory3DRenderOptions | None = None,
    progress: ProgressPortLike | None = None,
    progress_callback: Callable[[str], None] | None = None,
) -> FrameworkDynamicsRenderResult:
    ...
```

For a single topology, the renderer first calls `plot_decorated_graph_3d(...)` on the mean framework and then appends:

- one `Scatter3d` line trace per selected atom;
- grouped start and end marker traces, enabled by default and exposed as separate legend entries;
- atom, species, frame, frame ID, and time hover metadata.

When `show_start_end=True` (the default), the start-circle and end-diamond traces also appear in the legend whenever `show_legend=True`. They use independent legend groups so either endpoint class can be toggled without hiding the other.

Trajectory bounds are included in the final scene ranges. The result provides to `to_html(...)` and `write_html(...)` and maps atom indices to Plotly trace indices. The generic graph renderer remains unchanged and contains no trajectory semantics.

Category legend groups

Every framework and atomic-connectivity trace belonging to topology category *k* uses

```
legendgroup = "framework-topology:<k>"
```

and the layout uses

```
legend.groupclick = "toggllegroup"
```

One legend click therefore toggles the complete averaged framework and atomic mean-connectivity layer for that category. The dominant category is visible initially; other categories begin as `legendonly`. Global density and trajectory traces are not placed in topology-category legend groups.

The legend title reports topology ID, probability, frame count, and segment count. Trace construction is a hard contract. Partitioned scenes retain the validated cell wireframe and layout from the generic renderer, but category geometry is emitted by a compact adapter with at most four traces per topology class:

- one framework-edge trace;
- one framework-node trace with per-point colors;
- one atomic-connectivity edge trace;
- one atomic-position trace with per-point colors.

The general renderer's style buckets are not copied into every category. For seven classes the category contribution is therefore at most 28 traces, leaving room for densities, trajectories, endpoints, and the cell within the balanced 96-trace browser profile.

Interactive density scene fitting

Dense and sparse HDR shells are normalized to `DensityShellGeometry` and passed to `fit_density_scene_to_browser_budget` before Plotly mesh traces are created. The renderer owns assembly order and non-density trace reservation; it does not own mesh simplification or budget arithmetic. The default profile is balanced; compact, quality, and custom profiles are accepted.

Progress reporting

Both preparation and rendering accept the package-wide structured `progress=` port. Preparation reports scene setup, frame registration, trajectory preparation, atomic mean-graph aggregation, backend planning, and field realization. Rendering reports scene assembly and one X/Y item for each requested density shell. Nested atomic and framework density modules receive the same resolved port.

The former `progress_callback=Callable[[str], None]` remains a deprecated compatibility alias. The normative event schema and adoption rules are owned by `docs/specs/progress_spec.{md,pdf}`.

Resource policy

`FrameworkDynamicsResources` owns one complete-scene runtime budget in addition to trajectory-specific safeguards:

```
FrameworkDynamicsResources(  
    max_memory_bytes=None,  
    max_threads=None,  
    max_wall_time_seconds=None,  
    memory_fraction=0.80,  
    thread_fraction=0.90,  
)
```

Omitted compute limits resolve from the current process/job allocation. The default is 80% of detected available memory, 90% of detected CPUs, and a 1,200-second preparation plus rendering objective. API values override `MDSTATS_MAX_MEMORY_BYTES`, `MDSTATS_MAX_THREADS`, and `MDSTATS_MAX_WALL_TIME_SECONDS`; memory and thread requests are clamped to the detected runtime ceiling.

The scene budget is immutable and context-local. Trajectory, density, mesh, cache, and child-worker helpers inherit it exactly. Legacy low-level values may tighten but cannot expand it. Aggregate retained and transient memory and summed estimated wall time are checked before allocation. Browser faces, vertices, traces, and HTML bytes remain a separate client-output profile.

Trajectory-specific resolved safeguards include:

```
max_frames  
max_trajectory_atoms  
max_trajectory_points  
max_trajectory_traces
```

Preparation rejects oversized frame, atom, or point requests before allocating the complete path result. Rendering separately rejects excessive trajectory trace counts. Scientific path coordinates are never silently decimated. The complete LD10 policy is `density_runtime_resource_policy_ld10_spec.{md,pdf}`.

Failure semantics

GraphAdapterError malformed selections, frames, cells, registration options, graph identity, gauge alignment, or periodic winding.

TrajectoryRequiredError a trajectory overlay is requested from an independent ensemble.

GraphComplexityError a declared frame, atom, point, or trace resource limit is exceeded.

GraphVisualizationError Plotly import, HTML serialization, or file writing fails.

No failure causes topology reconstruction, atom remapping, path interpolation, or silent frame removal.

Numerical and semantic limitations

- Uniform frame weights are implemented. Explicit time or ensemble statistical weights are deferred.

- Framework registration removes translation only; rotational and affine alignment are deferred.
- A single FrameworkTopology must describe all selected frames only in single-topology mode. Reactive or thermally partitioned trajectories should supply a compatible TopologyCatalog.
- Folded paths show positions in one display cell and intentionally remove net lattice displacement from the picture; continuous paths retain it.
- The trajectory line is a piecewise-linear visual interpolation between saved frames. It is not a reconstruction of the unresolved motion between outputs.
- Per-atom trajectory rendering may create many Plotly traces when group_by_species=False; the default species-grouped mode remains bounded.

External methods and provenance

No nontrivial external scientific algorithm is introduced in Plot-D1/D2. The implementation uses project-owned periodic graph gauges, standard affine coordinate transforms, arithmetic averaging, and piecewise-linear rendering. Plotly is an optional rendering dependency and does not define scientific identity or geometry.

Focused validation

The focused test boundary must verify:

1. explicit atom and species selections resolve as one stable union;
2. trajectory overlays reject independent ensembles;
3. continuous paths preserve periodic crossings without false jumps;
4. folded paths insert explicit cell-boundary breaks;
5. material coordinates remove homogeneous cell scaling;
6. laboratory coordinates retain homogeneous scaling;
7. framework registration removes common framework and trajectory drift;
8. reference and mean display-cell policies are explicit;
9. resource limits fail before oversized scene construction;
10. mean graph identity and periodic edge winding are preserved;
11. Plotly traces, endpoints, hover metadata, and HTML serialization succeed;
12. existing framework-topology, periodic-graph, and generic 3-D renderer tests remain unchanged;
13. partitioned catalogs produce exact category frame populations and normalized probabilities;
14. dominant-category selection is deterministic;
15. one legend action toggles every framework and atomic-connectivity trace in a category;
16. global trajectories and density fields are not duplicated by category;
17. density shells are fitted before Plotly assembly and final HTML writing.

Plot-D3 and Plot-D4 density extensions

The framework-dynamics scene may now own three renderer-independent density channels prepared in the same registration gauge:

$$\rho_A(\mathbf{x}) = \sum_t w_t \sum_i \delta[\mathbf{x} - \mathbf{r}_i(t)],$$

$$\rho_V(\mathbf{x}) = \sum_t w_t \sum_v \delta[\mathbf{x} - \mathbf{r}_v(t)],$$

and the edge-length measure

$$\rho_E(\mathbf{x}) = \sum_t w_t \sum_e \int_0^{L_e(t)} \delta[\mathbf{x} - \gamma_{e,t}(s)] ds.$$

Atomic and framework-vertex fields have units of inverse volume. The framework edge field has units of inverse area and remains a separate scientific channel. Plot-D3 preparation and rendering are owned by `atomic_density_spec.md`; Plot-D4 vertex/edge measures, projected versus atom-resolved path policies, and their normalization are owned by `framework_density_spec.md`.

Example topology-inference boundary

The LTA plotting example must not classify framework topology with one instantaneous fixed cutoff. It first constructs a framework-only hysteretic Si/Al–O connectivity trajectory, builds the `TopologyCatalog` from that result, and computes the decorated atomic mean graph from a second hysteretic definition that additionally includes present Li/Na/K–O pairs. This separation prevents mobile-ion coordination changes from multiplying framework topology states and reduces thermal fragmentation of category segments.